big data and cognitive computing

MDPI

Article

Cognitive Computing with Large Language Models for Student Assessment Feedback

Noorhan Abbas and Eric Atwell *

Artificial Intelligence for Language Group, School of Computer Science, University of Leeds, Leeds LS2 9JT, UK; n.h.abbas@leeds.ac.uk
* Correspondence: e.s.atwell@leeds.ac.uk

Abstract: Effective student feedback is fundamental to enhancing learning outcomes in higher education. While traditional assessment methods emphasise both achievements and development areas, the process remains time-intensive for educators. This research explores the application of cognitive computing, specifically open-source Large Language Models (LLMs) Mistral-7B and CodeLlama-7B, to streamline feedback generation for student reports containing both Python programming elements and English narrative content. The findings indicate that these models can provide contextually appropriate feedback on both technical Python coding and English specification and documentation. They effectively identified coding weaknesses and provided constructive suggestions for improvement, as well as insightful feedback on English language quality, structure, and clarity in report writing. These results contribute to the growing body of knowledge on automated assessment feedback in higher education, offering practical insights for institutions considering the implementation of open-source LLMs in their workflows. There are around 22 thousand assessment submissions per year in the School of Computer Science, which is one of eight schools in the Faculty of Engineering and Physical Sciences, which is one of seven faculties in the University of Leeds, which is one of one hundred and sixty-six universities in the UK, so there is clear potential for our methods to scale up to millions of assessment submissions. This study also examines the limitations of current approaches and proposes potential enhancements. The findings support a hybrid system where cognitive computing manages routine tasks and educators focus on complex, personalised evaluations, enhancing feedback quality, consistency, and efficiency in educational settings.

Keywords: data science assessments; Mistral-7B; CodeLlama-7B; automatic assessment feedback; Python code analysis

check for updates

Academic Editors: Zuchao Li and Min Peng

Received: 18 February 2025
Revised: 7 April 2025
Accepted: 16 April 2025
Published: 23 April 2025

Citation: Abbas, N.; Atwell, E. Cognitive Computing with Large Language Models for Student Assessment Feedback. *Big Data Cogn. Comput.* **2025**, *9*, 112. https://doi.org/10.3390/bdcc9050112

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Assessment feedback plays a pivotal role in higher education, representing one of the most powerful influences on learning and achievement [1,2]. Effective feedback serves as a critical pedagogical tool that not only identifies areas for improvement but also reinforces successful learning strategies and achievements, thereby fostering students' self-regulated learning capabilities [3]. High-quality feedback highlights students' strengths, pinpoints areas requiring development, and provides constructive guidance for enhancement, facilitating the development of evaluative judgement and feedback literacy [4]. This feedback process is particularly crucial in technical disciplines where conceptual understanding must be paired with practical implementation skills. However, as Evans [1] notes, the effectiveness of feedback depends heavily on its timeliness, consistency, and students' ability to engage with and implement the received feedback.

The exponential growth in data science education enrolment has presented significant challenges for higher education institutions, particularly in providing timely and comprehensive feedback on student assessments [5]. As class sizes continue to expand, the traditional approach of manual assessment by multiple markers often results in variations in feedback quality and consistency, potentially impacting student learning outcomes [6]. This inconsistency in marking practices has been identified as a significant concern in maintaining academic standards and ensuring fair assessment [7]. While automated assessment systems have shown promise in addressing these challenges [8], concerns regarding data privacy, institutional control, and integration with existing infrastructure have limited their widespread adoption [9].

Proprietary models such as GPT-3.5 and GPT-4 have demonstrated some success in the area of generating feedback on student assignments. However, open-source alternatives are gaining traction due to their accessibility, flexibility, and cost-effectiveness [9]. Recent advancements in open-source LLMs present a promising avenue for developing automated feedback systems that address these concerns while maintaining institutional autonomy [8]. These models, when deployed within institutional infrastructure, offer the potential to generate detailed, consistent feedback while ensuring data sovereignty and compliance with privacy regulations. The open-source nature of these models allows for customisation and fine-tuning to specific educational contexts, potentially enhancing their effectiveness in evaluating domain-specific content in data science reports.

This paper focuses on the application of open-source models, specifically Mistral-7B [10] and CodeLlama-7B [11], for the automated generation of feedback on data science reports. These models were selected due to their demonstrated capabilities in processing both programming and narrative elements, making them particularly suitable for evaluating data science submissions. By utilising these models, the study aims to evaluate the quality, accuracy, and pedagogical value of the generated feedback compared to traditional manual feedback, providing insights into how AI can augment current assessment practices. Furthermore, the study addresses critical aspects, including feedback quality, consistency, accuracy, pedagogical value, and the necessity of human oversight in the assessment process. We report experimental results on a sample of assessment submissions for a postgraduate programming course. There are around 22 thousand assessment submissions per year in the School of Computer Science, which is one of eight schools in the Faculty of Engineering and Physical Sciences, which is one of seven faculties in the University of Leeds, which is one of one hundred and sixty-six universities in the UK, so there is clear potential for our methods to scale up to millions of assessment submissions. The findings contribute to the growing body of knowledge on automated assessment feedback in higher education, offering practical insights for institutions considering the implementation of open-source LLMs in their assessment workflows. Additionally, this study examines the limitations of current approaches, such as inconsistencies in feedback and the challenge of handling complex report components, and proposes potential enhancements through the integration of multimodal capabilities and advanced document processing frameworks.

The primary contribution of this paper is threefold: (1) we provide a comprehensive evaluation of open-source LLMs (Mistral-7B and CodeLlama-7B) for generating feedback on data science reports containing both code and narrative elements, demonstrating their effectiveness compared to traditional manual assessment; (2) we establish empirical benchmarks for feedback quality and consistency that can serve as calibration tools for human assessors in large-scale educational settings; and (3) we identify specific limitations and optimal configurations of these models for educational feedback generation, offering practical insights for institutions seeking to implement such systems while maintaining pedagogical integrity. By focusing on open-source models deployed within institutional infrastructure,

our approach addresses critical concerns regarding data sovereignty, privacy compliance, and institutional autonomy in automated assessment systems.

The remainder of this paper is structured as follows: Section 2 reviews the existing literature on AI-based feedback generation in educational settings. Section 3 describes the methodology used to implement and evaluate the open-source models. Section 4 presents the results of the comparative analysis between AI-generated and human-provided feedback, discusses the implications of the findings for educators, and suggests potential areas for future research in AI-assisted learning.

2. Background Research

The landscape of data science assessments is distinctively characterised by Jupyter Notebooks, a computational environment that integrates narrative markdown documentation with executable code cells [12]. This hybrid format presents a unique analytical challenge, as assessment evaluation requires a multifaceted approach that simultaneously examines both textual exposition and computational implementation [13]. Consequently, LLMs trained to process both textual and coding contexts are particularly well positioned to provide comprehensive analytical feedback on such multidimensional academic submissions. Building upon the unique compositional structure of Jupyter Notebooks, the existing literature provides emerging insights into the potential of LLMs for automated assessment across various educational domains. Recent research has explored the application of these models in diverse assessment contexts, offering valuable perspectives on their capabilities and limitations.

In the domain of programming education, MacNeil et al. [14] explored the use of the proprietary GPT-3 LLM to generate diverse explanations of code for educational purposes. Their study examines the potential of GPT-3 to effectively support students in learning introductory programming concepts in Python by providing varied and meaningful explanations. Through an analysis of over 700 prompts across multiple code snippets, the authors identified eight distinct types of explanations that GPT-3 can generate. These include tracing the execution of code, fixing bugs, generating analogies, listing relevant programming concepts, predicting console output, explaining time complexity, identifying common beginner mistakes, and summarising code at multiple levels of abstraction. The findings indicate that GPT-3 can generate diverse and contextually appropriate explanations that may enhance students' understanding of Python code. The study underscores the value of using LLMs to offer different perspectives on a given code segment, which can be particularly helpful for novice programmers. However, challenges remain, particularly concerning the need for effective prompt engineering to ensure explanations are pedagogically sound and consistent.

Viet and Markov [15] explored LLMs' capabilities in automated bug localisation and program repair for Python assignments. Their research utilised pre-trained models like Galpaca-30B and GPT-3.5-turbo, demonstrating improved accuracy in bug identification through few-shot prompting. Complementing this, Savelka et al. [16] conducted a comprehensive evaluation of GPT-4 across several assessment tasks in programming courses, revealing significant improvements over previous GPT versions in handling multiple-choice and coding assessments.

Dai et al. [17] explored the feasibility of employing ChatGPT (2023 version) for generating feedback on student assignments in a postgraduate data science course. Their study assessed the readability, agreement with human instructors, and the overall effectiveness of the feedback produced by ChatGPT. The results indicated that ChatGPT generated feedback that was more readable and consistent compared to instructor-provided feedback. Moreover, the model achieved a high level of agreement with instructors in evaluating the

appropriateness of assignment topics. However, ChatGPT exhibited limitations in delivering detailed process-focused and self-regulatory feedback, which are essential for guiding student learning. Another limitation of the study was the use of a proprietary model without utilising its API, implying that student data might be used in the pre-training of future models, thus violating the privacy rules of higher education institutions. Additionally, the paper does not specify whether the assessments contained any code, which is an important consideration when evaluating the applicability of feedback in data science education. The authors conclude that while ChatGPT shows considerable promise for automating feedback provision, careful prompt engineering and ongoing human oversight are necessary to enhance the quality and pedagogical effectiveness of AI-generated feedback.

In assessing the markdown text in data science reports, researchers have explored the use of LLMs to evaluate and provide feedback effectively. Although markdown text in data science reports differs from traditional essays in format and content, both require structured, meaningful feedback that addresses clarity, accuracy, and completeness. Yancey et al. [18] demonstrated the potential of GPT-3.5 and GPT-4 in rating short essays, particularly for L2 English learners, with GPT-4 showing comparable performance to established Automatic Writing Evaluation methods. Similarly, Xiao et al. [19] investigated LLMs in Automated Essay Scoring, proposing a human–AI collaborative framework that highlighted the models' potential to enhance grading efficiency and provide explainable feedback.

Venter, Coetzee, and Schmulian [20] investigated the integration of AI into feedback delivery to enhance learning in higher education. The study explored the use of a customised GPT-4 prompt to generate feedback for second-year accounting students enrolled in a competency-based intermediate accounting course in South Africa. The AI-generated feedback, which focused on essay-style questions, was delivered through a no-code web application developed using Bubble.io. The findings indicate that AI-generated feedback can align with established principles of effective feedback, such as providing clarity, actionable guidance, and fostering student self-reflection. However, the study also highlights inconsistencies in the quality of AI-generated feedback, emphasising the importance of prompt engineering and continuous human oversight to maintain quality and pedagogical relevance. The authors conclude that, despite existing challenges, AI tools like GPT-4 hold considerable promise for complementing traditional feedback methods, particularly in large-scale educational contexts.

These studies collectively underscore the transformative potential of LLMs in educational assessment, while also highlighting critical considerations of model performance, fairness, and the importance of human oversight. It also suggests that open-source models, when deployed within institutional infrastructures, can address data privacy concerns while providing transparent and controlled AI-assisted assessment capabilities [21]. Given the rapid progress in LLM research, this study is timely and relevant, proposing a robust approach that can be adapted to other datasets while laying the groundwork for future methodologies.

3. Materials and Methods

3.1. Dataset

The dataset for this study was derived from 15 summative assessment submissions in the form of data science project reports from Master's-level students at a British higher education institution. It is important to note that this research was designed and approved as a pilot study, with formal consent from the School of Computer Science and the Digital Education Service as well as the students who submitted the assessments, specifically, limiting our analysis to this small sample size. This controlled approach allowed us to thoroughly explore the methodological feasibility of using open-source LLMs for assess-

ment feedback generation while maintaining appropriate data governance protocols. The findings from this pilot study are intended to establish preliminary evidence and refine our approach before proceeding to a larger-scale investigation, for which approval has subsequently been secured. While the sample size is limited, it provides sufficient data to identify initial patterns in model performance and highlight areas requiring further investigation in the planned expanded study.

The assessment instrument was structured into four primary components: Project Plan (includes dataset description, project objectives, and system design), Program Code (implementation of programming solutions), Project Outcome (analysis of project outcomes and visualisations), and Conclusion (conclusive findings with presentation). Each component was further subdivided into specific subsections designed to evaluate distinct learning objectives and competencies. The Project Plan section comprised 11 rubric items, the Program Code section included 9 rubric items, the Project Outcome section contained 11 items, and the Conclusion section encompassed 4 items. Table 1 presents the character counts for each section of the data science project reports, providing an overview of the dataset size for each of the four primary components.

Table 1. Dataset size for each report section (in words).

	Project Plan	Program Code	Project Outcome	Conclusion	Total Words
Report 1	535	927	468	325	2255
Report 2	799	2086	521	393	3799
Report 3	1105	2650	1006	268	5029
Report 4	966	2135	1054	650	4805
Report 5	994	9078	1185	246	11,503
Report 6	1388	6298	1071	969	9726
Report 7	503	3378	545	376	4802
Report 8	959	3884	286	254	5383
Report 9	721	1082	561	252	2616
Report 10	1583	542	1475	473	4073
Report 11	956	3847	1359	665	6827
Report 12	2659	2622	2421	524	8226
Report 13	675	7486	1012	3448	12,621
Report 14	941	1112	553	584	3190
Report 15	978	2230	582	188	3978
Total	15,762	49,357	14,099	9615	88,833

The dataset size may be too small for statistical significance testing of the results, but it may be sufficient to give insights into the capabilities and limitations of LLMs for this task.

To ensure standardisation and facilitate assessment, students were provided with a templated Jupyter Notebook environment as the project development platform. This structured approach required students to document their methodology and findings using markdown syntax for textual content, while implementing their computational solutions within designated code cells. For assessment purposes, students were required to submit two files: a comprehensive PDF document containing the executed Notebook outputs, including all computational results and visualisations, alongside the original Jupyter Notebook file. The inclusion of the source Notebook enabled evaluators to validate results through code execution when necessary, ensuring assessment reliability and facilitating a detailed technical review of the implementations.

A stratified random sampling approach was employed to select 15 anonymised submissions from the corpus, ensuring representation across the spectrum of demonstrated competencies as indicated by their summative assessment scores. This sampling strategy was designed to capture diverse levels of technical proficiency and academic performance

within the student cohort. The selected submissions were subsequently analysed using two distinct LLMs: Mistral-7B-Instruct-v0.3 and CodeLlama-7B-Instruct. These models were deployed to generate evaluative feedback for each section and their corresponding subsections within the reports. The feedback generation process was systematically controlled through carefully crafted prompts, which instructed the models to identify and articulate both a primary strength and a critical area for improvement within each subsection. This binary feedback approach was designed to maintain consistency in the automated evaluation process while providing balanced constructive assessment across all components of the submissions.

3.2. Models

Codellama/CodeLlama-7b-Instruct-hf: Code Llama (7B-Instruct), developed by Meta AI [11], is a specialised 7-billion-parameter language model specifically trained for code comprehension and generation tasks. The model's architecture is built upon the Llama 2 foundation but optimised for programming tasks through extensive pre-training on code-specific datasets, including public GitHub repositories and programming-related documentation. The instruction-tuned variant (-Instruct-hf) is particularly adept at following specific prompts and generating contextually appropriate responses, making it well suited for providing feedback on data science assessments. It can run efficiently on consumer-grade GPUs with 16 GB VRAM, making it accessible for academic research environments. The model exhibits strong capabilities in analysing code structure, identifying programming patterns, and evaluating technical implementations across multiple programming languages, with particular strength in Python—the predominant language in data science. Its specialised training enables it to provide detailed feedback on code quality, algorithmic efficiency, and implementation correctness, while maintaining the ability to explain complex programming concepts in an educational context.

Mistral-7B-Instruct-v0.3: This model, developed by Mistral AI [10], is a 7-billion-parameter LLM based on an improved transformer architecture with extended vocabulary, function calling, and an advanced tokeniser. The model can run efficiently on consumer-grade GPUs with 16GB VRAM due to its optimised attention mechanisms and grouped-query attention, making it more accessible for academic research compared to larger models. While the exact pre-training dataset is not publicly disclosed by Mistral AI, the model incorporates a diverse range of high-quality internet text, academic papers, and programming resources, with particular emphasis on technical and scientific content. This makes it particularly suitable for providing feedback on data science assessments, as it demonstrates strong capabilities in understanding statistical concepts, programming syntax, and analytical methodologies. The instruction-tuned nature of v0.3 enables it to follow specific grading criteria and provide structured feedback, while its technical foundation allows it to identify errors in code implementation, statistical analysis, and methodology application.

GPT-4o-mini (https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/ (accessed on 7 April 2025)): This is OpenAI's latest compact model, which is designed to deliver high performance at a significantly reduced cost. It achieves an 82% score on the Massive Multitask Language Understanding benchmark, surpassing GPT-3.5 Turbo and other small models in both textual intelligence and multimodal reasoning. GPT-4o-mini is over 60% more affordable than GPT-3.5 Turbo, making advanced AI capabilities more accessible. The model supports text and vision inputs, with plans to include text, image, video, and audio inputs and outputs in the future. It features a context window of 128,000 tokens and can handle up to 16,000 output tokens per request, with knowledge up

to October 2023. GPT-4o-mini also demonstrates strong performance in function calling, enabling developers to build applications that interact with external systems.

3.3. Experimental Design

3.3.1. Preliminary Model Evaluation Phase

Our investigation employed a two-phase approach: an initial preliminary evaluation followed by a comprehensive assessment of selected models. The preliminary evaluation phase was designed to test the capabilities and limitations of three distinct language models—GPT-4o-mini, Mistral-7B, and CodeLlama-7B—for generating automated feedback on data science reports.

For this initial phase, we developed a synthetic assessment report to validate the feedback generation capabilities across different model architectures and computational requirements. GPT-4o-mini operated via API calls while Mistral-7B and CodeLlama-7B were deployed locally on an NVIDIA A100 GPU infrastructure. The synthetic report approach was specifically chosen to avoid data privacy issues while using GPT-4o-mini, as proprietary API-based models present inherent data sovereignty concerns in educational contexts.

Our initial experimental protocol implemented a holistic approach, processing complete reports without sectional partitioning and utilising comprehensive assessment rubric prompts. Multiple iterations were conducted to evaluate model performance and reliability. The experiments revealed that GPT-4o-mini successfully generated discriminative feedback, identifying both strengths and weaknesses across report sections, with optimal performance achieved using the following parameters:

- Chunk size: 1500 tokens;
- Overlap margin: 200 tokens;
- Batch processing: 3 chunks per iteration.

However, this preliminary testing revealed significant differences in model capabilities. While GPT-4o-mini demonstrated satisfactory performance with the holistic approach, both CodeLlama-7B and Mistral-7B exhibited limitations in processing comprehensive reports as single units. This divergence in model capabilities necessitated a methodological refinement, leading to the development of a sectional processing approach.

The experimental results from this phase, combined with institutional data privacy considerations, informed our decision to proceed exclusively with Mistral-7B and CodeLlama-7B for the primary evaluation phase. These open-source models could be deployed within the institution's infrastructure, ensuring data sovereignty and compliance with privacy regulations—a critical consideration for educational assessment systems when processing student data. Additionally, the preliminary phase allowed us to optimise our approach through the following:

1. Systematic partitioning of report content into discrete sections;
2. Development of section-specific prompts;
3. Independent processing of each section for targeted feedback generation.

This optimisation of the evaluation protocol not only addressed the limitations observed with CodeLlama-7B and Mistral-7B but also established a more structured and scalable approach to automated feedback generation. The sectional methodology subsequently served as the foundation for the full-scale experimental implementation described in the following section.

3.3.2. Primary Evaluation Phase

Following the preliminary evaluation, we conducted a comprehensive assessment of Mistral-7B and CodeLlama-7B using 15 actual student reports. This phase implemented

the sectional processing approach developed during preliminary testing, with several key methodological refinements.

For the primary evaluation, we systematically partitioned each report into four discrete components: Project Plan, Program Code, Project Outcome, and Conclusion. This structured segmentation allowed for the targeted analysis of specific assessment components while accommodating the context window limitations identified in the preliminary phase. Each section was processed independently with section-specific prompts designed to elicit detailed feedback on relevant assessment criteria.

The model parameters were carefully calibrated based on our preliminary findings to optimise feedback quality and consistency.

Temperature (τ = 0.2): A relatively low temperature value was selected to promote deterministic outputs, reducing random variations in feedback generation. This configuration ensures consistency across multiple evaluations while maintaining sufficient creativity to articulate diverse feedback points. The low temperature setting is particularly crucial in educational assessment contexts where the reliability and reproducibility of feedback are paramount.

Nucleus Sampling (top-p = 0.6): The nucleus sampling parameter was set to constrain the cumulative probability distribution of output tokens, effectively filtering out lower-probability outputs while maintaining natural language fluency. This moderate value strikes a balance between diversity and precision in feedback generation, ensuring that the generated content remains focused and relevant to the assessment criteria.

Maximum Token Length (max_new_tokens = 600): The token limit was established based on an empirical analysis of human-generated feedback samples, providing sufficient capacity for detailed feedback while maintaining conciseness. This parameter ensures comprehensive coverage of both strengths and weaknesses for each subsection while preventing overly verbose or redundant feedback.

These hyperparameters were determined through iterative experimentation and validated against a development set of student submissions to optimise the trade-off between feedback quality and computational efficiency. The configuration demonstrates robust performance across diverse assessment components while maintaining consistency with human evaluation patterns.

The feedback generation process was structured to maintain consistency by instructing the models to identify and articulate both a primary strength and a critical area for improvement within each subsection. This binary feedback approach was designed to maintain consistency in the automated evaluation process while providing balanced constructive assessment across all components of the submissions.

To facilitate systematic evaluation, we developed a standardised prompt template for each report section, with explicit instructions to focus on specific assessment criteria relevant to that section. For example, prompts for the Program Code section directed the models to evaluate code structure, error handling mechanisms, and documentation quality, while prompts for the Project Outcome section emphasised visualisation effectiveness and analytical depth.

The generated feedback was subsequently evaluated against human-provided feedback (considered as ground truth) using both quantitative metrics (precision, recall, and F1-score) and qualitative assessments of feedback coherence, consistency, style, structure, and completeness. This comprehensive evaluation approach allowed us to assess both the technical accuracy and pedagogical value of the feedback generated by each model.

3.4. Report Processing and Content Extraction

The implementation of automated feedback generation necessitated a robust approach to process and extract content from student report submissions. Our pipeline consisted of three key components: document ingestion, text extraction, and structured content organisation.

3.4.1. PDF Text Extraction Methodology

Initial attempts to implement text extraction utilising the PyPDF2 library revealed significant limitations in handling diverse PDF formatting structures present in student submissions. Following a comprehensive evaluation of alternative solutions, PDFPlumber was identified as a more robust extraction tool, demonstrating superior capability in processing heterogeneous PDF formats while maintaining content integrity.

3.4.2. Content Extraction Architecture

The implemented extraction pipeline employed PDFPlumber's advanced parsing algorithms to process individual pages sequentially, ensuring the complete capture of textual content while preserving structural relationships within the documents. This approach was particularly important for maintaining the integrity of code segments and their associated documentation.

3.4.3. Semantic Segmentation, Content Organisation, and Validation

Following successful text extraction, a sophisticated segmentation algorithm was developed to partition the content into the predefined assessment components: Project Plan, Program Code, Project Outcome, and Conclusive Presentation. The segmentation methodology employed regular expression pattern matching combined with hierarchical content classification to identify section demarcations and their associated content blocks.

The final phase of the extraction pipeline implemented a validation mechanism to ensure the structural integrity of the extracted content. This process encompassed the following:

1. Verification of section completeness;
2. Validation of hierarchical relationships between sections and subsections;
3. Content attribution to appropriate assessment components;
4. Preservation of formatting elements crucial for assessment.

This systematic approach to report processing facilitated subsequent automated analysis and feedback generation by providing a standardised, structured representation of student submissions. The robustness of this methodology was validated across the entire corpus of student submissions, demonstrating consistent performance regardless of variations in document formatting or structure.

3.5. Performance Metrics and Analysis

To evaluate the efficacy of the automated feedback generation systems, we employed a comprehensive set of quantitative metrics, comparing the LLM-generated feedback against expert human evaluations as the ground truth. The evaluation framework encompassed both traditional classification metrics and domain-specific measures to assess feedback quality.

3.5.1. Evaluation Metrics

Precision (P) was calculated at both micro and macro levels to evaluate the automated feedback accuracy as shown in Equations (1) and (2) below. Micro-precision (*P_micro*)

considers each individual feedback point across all reports, while macro-precision (*P_macro*) calculates precision for each report separately and then averages the results:

$$\boldsymbol{P_{micro}} = \frac{\boldsymbol{TP}}{\boldsymbol{TP} + \boldsymbol{FP}} \tag{1}$$

$$\boldsymbol{P_{macro}} = \frac{1}{n} * \sum_{i=1}^{n} \frac{TP_i}{TP_i + FP_i} \tag{2}$$

where *TP* represents true positives (correctly identified feedback points), *FP* represents false positives (incorrectly identified feedback points), *n* is the number of reports, and *i* represents each individual essay. Micro-precision provides an overall measure of accuracy across all feedback points, while macro-precision gives equal weight to each report regardless of the number of feedback points it contains.

Recall (R) was measured at both micro and macro levels to evaluate the completeness of automated feedback as shown in Equations (3) and (4). Micro-recall (*R_micro*) considers each individual feedback point across all reports, while macro-recall (*R_macro*) calculates recall for each report separately and then averages the results:

$$\boldsymbol{R_{micro}} = \frac{\boldsymbol{TP}}{\boldsymbol{TP} + \boldsymbol{FN}} \tag{3}$$

$$\boldsymbol{R_{macro}} = \frac{1}{n} * \sum_{i=1}^{n} \frac{TP_i}{TP_i + FN_i} \tag{4}$$

where *FN* represents false negatives (missed feedback points), *n* is the number of reports, and *i* represents each individual report. Micro-recall provides an overall measure of completeness across all feedback points, while macro-recall gives equal weight to each report regardless of the number of feedback points it contains.

F1-score was computed at both micro and macro levels to provide balanced measures between precision and recall as shown in Equations (5) and (6):

$$\boldsymbol{F1_{micro}} = \boldsymbol{2}\frac{\boldsymbol{P_{micro}} * \boldsymbol{R_{micro}}}{\boldsymbol{P_{micro}} + \boldsymbol{R_{micro}}} \tag{5}$$

$$\boldsymbol{F1_{macro}} = \frac{\boldsymbol{1}}{\boldsymbol{n}} * \sum_{\boldsymbol{i=1}}^{\boldsymbol{n}} \frac{\boldsymbol{2} * (\boldsymbol{P_i} * \boldsymbol{R_i})}{(\boldsymbol{P_i} + \boldsymbol{R_i})} \tag{6}$$

where *P* represents precision, *R* represents recall, *n* is the number of reports, and *i* represents each individual report. *Micro F*1 provides a balanced measure across all feedback points, while *macro F*1 gives equal weight to each report regardless of the number of feedback points it contains.

3.5.2. Qualitative Analysis

The qualitative analysis was conducted to evaluate the comprehensiveness and quality of the feedback generated by both the Mistral-7B and CodeLlama-7B models. The analysis focused on three key dimensions.

First, we examined feedback coherence and consistency across the reports, identifying instances where models provided contradictory or inconsistent feedback. As documented in our findings, CodeLlama-7B occasionally generated identical content as both a strength and a weakness within the same evaluation section, compromising the overall coherence of the feedback.

Second, we analysed the style and structural characteristics of the feedback, comparing how each model organised and presented information. We found that Mistral-7B typically employed a more structured approach with a clear categorisation of strengths and areas for

improvement, while CodeLlama-7B tended to provide more succinct feedback without the same level of organisation.

Third, we assessed the completeness and specificity of the feedback by comparing how each model addressed the same report sections. This comparison revealed that Mistral-7B generally provided more detailed, contextual feedback with concrete examples and specific suggestions for improvement (such as recommendations for figure legibility and scaling), whereas CodeLlama-7B offered broader, more generic recommendations (such as suggestions for more varied visualisations) without specific implementation guidance.

These qualitative assessments were supported by direct comparisons of feedback samples from both models across multiple report sections, including Data Description, Program Code, and Visualisation of Results, allowing us to draw conclusions about each model's relative strengths in providing pedagogically effective feedback.

4. Results and Discussion

4.1. Computing Infrastructure

The experimental implementation was conducted using Google Colab's cloud computing platform, leveraging an NVIDIA A100 GPU with 40 GB VRAM. This infrastructure choice was motivated by the need for efficient processing of LLMs while maintaining cost-effectiveness for academic research. The A100's architecture, particularly its Tensor Core technology, provides optimal performance for transformer-based models, enabling rapid inference for feedback generation. The system is implemented using the open-source framework PyTorch.

4.2. Preliminary Model Evaluation Results

4.2.1. Model Performance Comparison

The preliminary evaluation using our synthetic assessment report revealed distinct performance patterns across the three tested models. GPT-4o-mini demonstrated robust performance with minimal prompt engineering requirements, producing well-structured outputs that required no post-processing. The model effectively processed complete reports using our chunking strategy (1500 tokens with a 200-token overlap), successfully generating discriminative feedback that identified both strengths and weaknesses across all report sections.

In contrast, both open-source models (CodeLlama-7B and Mistral-7B) exhibited higher sensitivity to prompt formulation, mirroring the results reported by [22,23], and showed limitations in processing comprehensive reports as single units. When presented with complete reports, these models frequently produced inconsistent outputs, struggled with maintaining coherence across multiple assessment criteria, and occasionally failed to complete feedback generation tasks within reasonable computational constraints.

4.2.2. Comparative Prompt Sensitivity

The open-source models presented several specific challenges that required systematic resolution. A notable issue emerged in the models' interpretation of programming-related content, particularly in distinguishing between different types of classes in Python code analysis. For example, the models initially failed to differentiate between pandas DataFrame class references (generated when you run the code) and user-defined classes, requiring more explicit prompt engineering to achieve accurate assessment.

The preliminary testing revealed important differences in prompt sensitivity between proprietary and open-source models. While GPT-4o-mini responded effectively to generalised assessment prompts, CodeLlama-7B and Mistral-7B required more detailed, structured prompting with explicit boundary conditions and context-specific disambiguation rules.

For instance, the prompt structure that yielded optimal results for GPT-4o-mini was significantly simpler than those required for the open-source models. The example below illustrates this difference:

Gpt-4o-mini model prompt

Prompt_ProgramCode = (

The following text is from the "Program Code" section of a project report. Please use the following criteria to identify strengths and weaknesses in the text:

Criteria:

1. Program Code

- Imported packages/modules used appropriately

- Error handling and exception handling

)

On the other hand, the equivalent prompt utilised for the CodeLlama-7B and Mistral-7B Models was as follows:

prompt_ProgramCode = (

You are an expert in Python programming, technical writing and documentation analysis. Your task is to critically analyse the quality, clarity, and effectiveness of how this python code is written and structured. Focus on evaluating the presentation, specifically checking for the inclusion of user-defined functions, user-defined classes NOT the built-in ones. Notably, assess the logical flow, documentation standards, and whether claims are supported by evidence.

IMPORTANT RULES:

- Evaluate HOW the Python code is presented

- Focus on writing clarity, logical flow, and documentation standards

- Analyse whether claims are properly supported with evidence

- Assess the depth and rigor of technical explanations

- Evaluate detail level

Evaluate the following criteria:

Criteria:

1. Program Code

- Ensure that error handling and exception handling mechanisms are implemented.

- Verify the proper usage and documentation of imported libraries

- Focus specifically on user-defined classes and user-defined functions and not built-in ones from libraries

REQUIRED OUTPUT FORMAT:

For each criterion, provide:

✓Strength: [One clear strength with precise detail in your own words]

△ Area for Improvement: [One clear weakness in your own words]

Remember:

- Use analytical language that evaluates rather than describes

- Focus on insights, not summaries

- Be specific but DO NOT quote the original text

)

Output Processing Requirements and Methodological Adaptations

While GPT-4o-mini generated clean, focused feedback, the outputs from CodeLlama-7B and Mistral-7B frequently contained artefacts requiring post-processing such as remnant text from input documents, embedded prompt fragments, and extraneous contextual information.

To address these challenges in the following testing phases, the following methodological refinements were implemented:

1. Development of explicitly structured prompts with clear boundary conditions;
2. Implementation of context-specific disambiguation rules;
3. Integration of post-processing pipelines for output standardisation.

These findings highlight the importance of model-specific prompt engineering strategies and the need for robust post-processing protocols when implementing automated feedback systems using open-source language models. The variations in model behaviour suggest that while open-source models can achieve comparable performance to proprietary alternatives, they require more sophisticated prompt engineering and output processing frameworks.

4.3. Quantitative Evaluation of Primary Models

In this phase, a sample of 15 reports was utilised to evaluate the feedback generation capabilities of the Mistral-7B and CodeLlama-7B models. The performance metrics for both models are summarised in Tables 2 and 3.

Table 2. The Mistral-7B performance evaluation of feedback generation.

	Micro			Macro		
	Precision	**Recall**	**F1-Score**	**Precision**	**Recall**	**F1-Score**
Project Plan	0.84	0.87	0.85	0.78	0.87	0.85
Program Code	0.97	0.89	0.93	0.84	0.84	0.92
Project Outcome	0.87	0.89	0.88	0.82	0.89	0.88
Conclusion	1.00	0.98	0.99	0.93	0.98	0.99

Table 3. The CodeLlama-7B performance evaluation of feedback generation.

	Micro			Macro		
	Precision	**Recall**	**F1-Score**	**Precision**	**Recall**	**F1-Score**
Project Plan	0.83	0.72	0.77	0.83	0.71	0.80
Program Code	0.97	0.90	0.93	0.78	0.72	0.81
Project Outcome	0.90	0.84	0.87	0.90	0.84	0.92
Conclusion	1.00	0.98	0.99	1.00	1.00	1.00

In the Program Code evaluation, a more substantial difference emerged between the two models. While both achieved identical micro F1-scores of 0.93, their macro-level performances differed significantly. Mistral-7B maintained strong performance with a macro F1-score of 0.92, whereas CodeLlama-7B's performance dropped considerably to 0.81. This disparity, particularly evident in CodeLlama-7B's lower macro-precision (0.78) and macro-recall (0.72), suggests that Mistral-7B provides more consistent and reliable feedback across individual reports for Program Code evaluation.

In the Project Outcome section, both models demonstrated robust performance, which is attributable to the section's systematic organisational structure. The content is methodically arranged according to project objectives, with each objective following a consistent presentational pattern: beginning with analytical findings derived from data analysis, followed by corresponding visualisations accompanied by interpretative commentary. This highly structured format, characterised by its predictable sequence and clear delineation of components, appears to facilitate more effective processing by the language models.

The standardised layout potentially reduces the cognitive complexity of the feedback generation task, which is reflected in the performance metrics of both models. Mistral-7B achieved a slightly higher micro F1-score (0.88 vs. 0.87), indicating better

overall accuracy, while CodeLlama-7B demonstrated a marginally higher macro F1-score (0.92 vs. 0.88). This suggests that while Mistral-7B's aggregate performance was marginally superior, CodeLlama-7B exhibited more consistent performance across individual reports. These results indicate that the structured nature of the Project Outcome section enables both models to reliably identify and evaluate key elements within this systematically organised framework.

In the Conclusion section, both models achieved exceptional results with identical micro-metrics (precision: 1.00; recall: 0.98; F1-score: 0.99). CodeLlama-7B demonstrated slightly better macro-level performance with perfect scores across all metrics (precision: 1.00; recall: 1.00; F1-score: 1.00) compared to Mistral-7B's still impressive but slightly lower macro-precision (0.93). It is pertinent to note that the Conclusion section comprises purely descriptive text, focusing on project achievements, limitations, and future directions, rather than containing programmatic elements. This distinction in content type may contribute to the notably high performance metrics observed in this section, as natural language processing models are typically well suited to analysing narrative text that follows conventional document structure patterns. The absence of technical code segments potentially reduces the complexity of the feedback generation task in this section, as the models are not required to evaluate syntactic and semantic elements specific to programming languages.

In summary, the comparative analysis reveals distinct performance patterns between the two models across report sections. Mistral-7B demonstrated more consistent performance across all sections, with particularly strong results in Program Code (micro F1: 0.93; macro F1: 0.92) and robust performance in Project Plan evaluation (micro F1: 0.85; macro F1: 0.85). While CodeLlama-7B achieved comparable results in Project Outcome (micro F1: 0.87; macro F1: 0.92) and Conclusion sections (micro F1: 0.99; macro F1: 1.00), its performance notably declined in Project Plan (micro F1: 0.77; macro F1: 0.80) and showed inconsistency in Program Code sections (micro F1: 0.93; macro F1: 0.81). These performance patterns suggest that Mistral-7B offers more reliable and consistent feedback generation across all report sections, making it potentially more suitable for comprehensive report evaluation where uniform performance across different documentation components is essential.

Models' Context Window Size Limitations

The analysis revealed noteworthy limitations in CodeLlama-7B's ability to generate feedback for Python code segments within three reports. Further investigation determined that these particular cases contained code sections substantially longer than the normative length observed across the remaining reports. Specifically, the typical word range for code submissions spanned 542 to 3847 words, whereas the three anomalous reports exhibited word lengths of 6298, 7486, and 9078, respectively.

This disparity in code section length appears to have exceeded the contextual processing capabilities of the CodeLlama-7B model. In contrast, the Mistral-7B-Instruct-v0.3 model implements an extensive context window of 32,768 tokens (32k), which is facilitated through advanced attention mechanisms including sliding window attention and grouped-query attention. Comparatively, the CodeLlama-7b-Instruct-hf model employs a more constrained, yet still substantial, context window of 16,384 tokens (16k), which surpasses the capabilities of traditional transformer architectures.

These considerable context window dimensions prove particularly advantageous in the analysis of academic submissions, as they minimise the need for document segmentation while preserving semantic coherence across extended text. Mistral-7B's expanded context window affords additional capacity for processing lengthier code submissions, whereas CodeLlama-7B's window dimensions remain sufficient for typical programming

assignments and associated documentation. This distinction highlights the importance of contextual modelling in effectively evaluating diverse document formats, especially those containing lengthy technical components.

4.4. Qualitative Feedback Analysis

4.4.1. Evaluation of Feedback Coherence and Consistency

In general, the level of detail and comprehensiveness provided by the Mistral-7B model was more consistent across the reports compared to the CodeLlama-7B model. For instance, it was observed that CodeLlama-7B occasionally generated identical feedback as both a strength and a weakness, as illustrated by the following example:

CodeLlama-7B Output:

1. Description of Data:

Strength: The dataset description is clear and complete, providing all the necessary information about the dataset, including the source, data types, and size. The comments on accuracy, reliability, and data quality are also provided.

Area for Improvements:

** The dataset description could be more specific about the variables included in the dataset and their definitions.*

** The comments on data accuracy and reliability could be more detailed and provide examples to support the claims.*

Mistral-7B Output:

1. Data Description

- Strength: The dataset description is clear and comprehensive, providing a detailed explanation of the dataset's source, contents, and purpose. The author also explains the quality and accuracy of the data, which is helpful for understanding the dataset' s reliability.

- Area for Improvements: The author could provide more specific details about the data collection process, such as sampling methods or data cleaning techniques used. Additionally, the author could clarify the meaning of some variables, like "XXX," to make the description more accessible to readers.

4.4.2. Evaluating Pedagogical Effectiveness in Automated Feedback Generation: Style, Structure, and Completeness

In addition, the feedback provided by Mistral-7B and CodeLlama-7B differs significantly in both style and completeness. Mistral's feedback employs a clear and direct language, with a structured approach that effectively categorises strengths and areas for improvement. The tone is notably pedagogical, as it provides concrete examples for suggested enhancements. In contrast, CodeLlama's feedback is more succinct and less descriptive, providing a general overview rather than an in-depth critique. The writing style lacks the nuance and detail found in Mistral's analysis, offering broad suggestions without much elaboration or contextual examples, as illustrated by the following example from the Program Code section:

Mistral-7B

1. Program Code

✓Strength: The code imports the necessary libraries and initializes the dataframe. It also provides a function to filter data by interval, which is useful for cleaning and preprocessing data.

▲ *Area for Improvements: The code does not handle exceptions or errors effectively. For example, if the CSV file is not found or is not in the correct format, the code will crash without any error message. Additionally, the code does not provide any documentation for the functions it defines, making it difficult for others to understand their purpose and usage.*

2. Code Structure:

✓ *Strength: The code is organized by objective, with separate sections for loading data, cleaning data, and analyzing data. This makes it easy to follow the logic of the code.*

✧ *Area for Improvements: The functions for cleaning and filtering data are defined within the main code block, rather than being separated into their own modules. This makes the code less modular and harder to reuse in other projects.*

CodeLlama-7B

Criteria 1: Program Code

✓ *Strong: The code is well-structured and easy to read. The use of user-define functions and classes makes the code more modular and reusable.*

▱ *Area for Improvement: The code could benefit from more error handling and proper exception handling mechanism to ensure robustness and reliability.*

Criteria 2: Code Structure

✓ *Strong: The notebook is well-organized and easy to follow. The use of functions and classes helps to keep the code modular and maintainable.*

▱ *Area for Improvement: The code could benefit from more comments and documentation to make it more self-explanatory and easier to understand.*

Another example from the Project Outcome section is as follows:

Mistral-7B Output

2. Visualization of Results

Strength: The report includes visualizations for each objective and uses a variety of appropriate visualization types. The visualizations are informative and easy to understand, with clear titles, labels, and legends.

Area for Improvements: The report could improve by ensuring all writing on figures are legible, and that figures are appropriately sized and scaled. Additionally, providing a brief description of each figure within the report would improve the overall flow and readability of the report.

CodeLlama-7B Output

Criteria 2: Visualization

✓ *Strong: The visualization of the results is clear and informative, with appropriate visualizations for each objective and a clear and concisely written explanation of the visualization.*

✓ *Area for Improvement: The visualizations could be more varied and include more visualizations for different objectives, and the visualization could be more interactive and dynamic.*

In the feedback example shown above, CodeLlama-7B and Mistral-7B present distinct approaches in their feedback quality, with each model excelling in different aspects of evaluation. CodeLlama-7B offers concise, broad-level feedback that emphasises content variety and interactivity in visualisations. However, its recommendations remain somewhat generic, lacking the specificity needed for practical implementation. In contrast, Mistral-7B

delivers more detailed and actionable suggestions, with a particular focus on practical improvements such as figure legibility, appropriate scaling, and descriptive content. This depth of feedback makes Mistral-7B more effective in addressing technical and structural aspects of visual presentation. Consequently, Mistral-7B's output appears more suitable for contexts requiring precise, actionable guidance, while CodeLlama-7B's approach may be more appropriate for initial, high-level content review.

5. Conclusions

This study has demonstrated the capabilities and limitations of open-source models, specifically Mistral-7B and CodeLlama-7B, in automating feedback generation for data science reports that contain both programming and narrative elements. The evaluation of 15 sample reports revealed that Mistral-7B consistently outperformed CodeLlama-7B in providing detailed and actionable feedback across multiple report sections, particularly in areas such as Project Plan and Program Code, where precision and thorough analysis are crucial. For instance, in the Project Plan section, Mistral-7B achieved a micro F1-score of 0.85 compared to CodeLlama-7B's 0.77, demonstrating its superior ability to capture relevant feedback points. In the Program Code evaluation, both models achieved identical micro F1-scores of 0.93, but Mistral-7B's macro F1-score was significantly higher at 0.92 compared to CodeLlama-7B's 0.81, indicating greater consistency across individual reports. Mistral-7B's superior performance in these sections, highlighted by higher recall and F1-scores, suggests its greater aptitude for understanding and assessing complex project components, bridging both theoretical and technical aspects effectively.

In contemporary higher education institutions, large class sizes necessitate the allocation of multiple assessors for student evaluation. Despite the use of standardised rubrics, significant variability often persists in the quality, depth, and consistency of feedback provided by different markers [6]. The findings of this research demonstrate that the developed feedback models can serve as effective guiding templates for assessors, enhancing the uniformity and pedagogical value of assessment feedback. These empirically derived models function as calibration tools, establishing benchmarks for feedback quality and promoting greater consistency in both formative and summative assessment practices across marking teams.

Despite the advantages observed, the study also uncovered important limitations of both models. CodeLlama-7B struggled with lengthy Python code segments due to its constrained context window size, while Mistral-7B demonstrated greater robustness in handling extensive content thanks to its larger context capacity. Furthermore, the models exhibited inconsistencies in feedback coherence and completeness, with Mistral-7B generally providing more structured, pedagogically valuable feedback compared to the often succinct and less descriptive output from CodeLlama-7B.

The analysis of model parameters, including temperature, nucleus sampling, and maximum token length, highlighted the importance of careful calibration to optimise feedback quality while balancing computational efficiency. Both models showed promise in reducing the time burden on educators, with Mistral-7B, in particular, offering reliable performance that aligns closely with human evaluators. However, the necessity of human oversight remains paramount to ensure feedback quality and pedagogical relevance, especially in the context of educational assessment.

6. Limitations and Future Work

A significant methodological constraint identified in this study pertains to the sequential analysis of discrete report sections. Students occasionally present relevant content in sections other than those prescribed by the report structure, necessitating human oversight

to ensure comprehensive evaluation. This cross-sectional content displacement poses challenges for automated assessment processes, as pertinent information may be distributed across multiple sections rather than confined to their designated locations. While current limitations necessitate human verification, future research utilising larger language models with expanded context windows could potentially mitigate this constraint by enabling a holistic analysis of entire reports as unified documents. Such advancement could facilitate a comprehensive evaluation of content regardless of its location within the report structure. Nevertheless, in the current implementation, human verification remains essential to capture and appropriately evaluate content that deviates from the expected structural framework. This finding underscores the present necessity for a hybrid approach combining automated analysis with human expertise to ensure an accurate and thorough assessment of student work.

Furthermore, the current implementation, utilising Mistral-7B and CodeLlama-7B models, exhibits limitations in the extraction and processing of graphical elements from PDF documents, as these models lack image processing capabilities. To address this constraint, future research could explore the integration of more sophisticated document parsing frameworks, such as the Unstructured library, which offers enhanced capabilities for processing multimodal content. Moreover, the implementation of multimodal language models could potentially enable a comprehensive analysis of both textual and graphical components within academic submissions. This proposed enhancement would facilitate a more holistic evaluation of student work, particularly in disciplines where visual representations constitute critical elements of academic communication.

Given the inherent context window constraints of LLMs, further research is warranted to explore the decomposition of comprehensive prompts into multiple, criteria-specific prompts. This approach, leveraging few-shot learning with targeted examples for each criterion, may enhance the granularity and depth of generated feedback. Such segmentation could potentially circumvent context limitations while producing more detailed and nuanced outputs.

Inspired by the research conducted by Xu et al. [24], preliminary investigations were conducted to explore the efficacy of self-reflective prompting techniques in enhancing the models' ability to identify and rectify potential errors in their output. These techniques involved implementing additional prompting steps where the models were instructed to review their initial feedback, critically evaluate its accuracy and completeness, and make necessary refinements. The self-reflective prompts encouraged the models to validate their responses against specific criteria and assessment rubrics, potentially leading to more precise and comprehensive feedback generation. However, further comprehensive empirical evaluation is necessary to validate the effectiveness of this methodological approach. Future research should focus on systematic testing of these self-reflection mechanisms to establish their reliability and potential impact on feedback accuracy, including quantitative analysis of improvements in precision and recall metrics following the application of self-reflective prompting.

Finally, building upon the methodological foundation established in this study, several promising research avenues could extend our work. Future investigations might explore the application of reinforcement learning techniques to fine-tune feedback generation models based on instructor evaluations, potentially creating adaptive systems that progressively align with institutional assessment standards. Additionally, longitudinal studies could examine the impact of automated feedback on student learning outcomes and report quality over time, providing insights into the pedagogical effectiveness of these systems. There is also significant potential in developing domain-specific adaptations of our framework, customised for specific student groups such as mature students [25],

or for discipline-specific assessment criteria such as text translations [26] or case study reports [27]. Finally, comparative analyses between different model architectures could yield valuable insights into the optimal balance between computational efficiency and assessment quality, particularly in resource-constrained educational environments.

7. Ethical Considerations

The implementation of LLMs for assessment feedback in data science education presents multifaceted ethical considerations that warrant careful examination. A primary concern lies in the potential inconsistency of generated feedback and the phenomenon of hallucination, where models may produce plausible but factually incorrect responses [28–30]. These issues could significantly impact the reliability and validity of assessment feedback, potentially compromising educational equity and student learning outcomes. Furthermore, the risk of toxic or inappropriate content generation, albeit minimal in educational contexts, necessitates robust filtering mechanisms and human oversight. The integration of these technologies must therefore incorporate comprehensive bias mitigation strategies to ensure fair and equitable treatment across diverse student populations.

Data security and student privacy emerge as critical ethical imperatives in the deployment of LLMs within higher education institutions [21]. The processing and storage of student submissions, alongside the generation and maintenance of feedback data, require stringent protective measures to comply with educational data protection regulations and institutional privacy policies. Additionally, inherent biases in model training data may disproportionately affect certain student demographics, necessitating careful consideration of bias mitigation strategies in both model selection and implementation protocols [9]. These considerations underscore the importance of developing transparent frameworks for LLM deployment in educational assessment, ensuring accountability while maintaining the integrity of the academic feedback process.

Author Contributions: Conceptualisation, N.A. and E.A.; methodology, N.A. and E.A.; software, N.A.; validation, N.A.; formal analysis, N.A. and E.A.; investigation, N.A. and E.A.; resources, N.A.; data curation, N.A.; writing—original draft preparation, N.A.; writing—review and editing, N.A. and E.A.; project administration, N.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: Our initial data-set of student assessment submissions is unavailable to other researchers due to ethical and privacy restrictions. In future work we plan to collect a larger data-set and make this accessible to others via the Research Data Leeds Repository https://archive.researchdata.leeds.ac.uk/ (accessed on 7 April 2025).

Conflicts of Interest: The authors declare no conflict of interest.

Abbreviations

The following abbreviation is used in this manuscript:

LLMs Large Language Models

References

1. Evans, C. Making sense of assessment feedback in higher education. *Rev. Educ. Res.* **2013**, *83*, 70–120. [CrossRef]
2. Harris, L.R.; Brown, G.T.L.; Harnett, J.A. Understanding classroom feedback practices: A study of New Zealand student experiences, perceptions, and emotional responses. *Educ. Assess. Eval. Account.* **2014**, *26*, 107–133. [CrossRef]
3. Nicol, D.J.; Macfarlane-Dick, D. Formative Assessment and Self-Regulated Learning: A Model and Seven Principles of Good Feedback Practice. *Stud. High. Educ.* **2006**, *31*, 199–218. [CrossRef]

4. Carless, D.; Boud, D. The development of student feedback literacy: Enabling uptake of feedback. *Assess. Eval. High. Educ.* **2018**, *43*, 1315–1325. [CrossRef]
5. Leiker, D.; Finnigan, S.; Gyllen, A.R.; Cukurova, M. Prototyping the use of large language models (llms) for adult learning content creation at scale. *arXiv* **2023**, arXiv:2306.01815.
6. Boud, D.; Molloy, E. Rethinking models of feedback for learning: The challenge of design. *Assess. Eval. High. Educ.* **2013**, *38*, 698–712. [CrossRef]
7. Bloxham, S.; Den-Outer, B.; Hudson, J.; Price, M. Let's stop the pretence of consistent marking: Exploring the multiple limitations of assessment criteria. *Assess. Eval. High. Educ.* **2016**, *41*, 466–481. [CrossRef]
8. Yan, L.; Sha, L.; Zhao, L.; Li, Y.; Martinez-Maldonado, R.; Chen, G.; Li, X.; Jin, Y.; Gašević, D. Practical and ethical challenges of large language models in education: A systematic scoping review. *Br. J. Educ. Technol.* **2024**, *55*, 90–112. [CrossRef]
9. Kasneci, E.; Sessler, K.; Küchemann, S.; Bannert, M.; Dementieva, D.; Fischer, F.; Gasser, U.; Groh, G.; Günnemann, S.; Hüllermeier, E.; et al. ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education. *Learn. Individ. Differ.* **2023**, *103*, 102274. [CrossRef]
10. Jiang, A.Q.; Sablayrolles, A.; Mensch, A.; Bamford, C.; Chaplot, D.S.; Casas, D.D.L.; Bressand, F.; Lengyel, G.; Lample, G.; Saulnier, L.; et al. Mistral 7B. *arXiv* **2023**, arXiv:2310.06825.
11. Roziere, B.; Gehring, J.; Gloeckle, F.; Sootla, S.; Gat, I.; Tan, X.E.; Adi, Y.; Liu, J.; Sauvestre, R.; Remez, T.; et al. Code llama: Open foundation models for code. *arXiv* **2023**, arXiv:2308.12950.
12. Kluyver, T.; Ragan-Kelley, B.; Pérez, F.; Granger, B.E.; Bussonnier, M.; Frederic, J.; Jupyter Development Team. Jupyter Notebooks—A publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*; IOS Press: Amsterdam, The Netherlands, 2016; pp. 87–90.
13. Rule, A.; Tabard, A.; Hollan, J.D. Exploration and explanation in computational notebooks. In Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems, Montreal, QC, Canada, 21–26 April 2018; pp. 1–12.
14. MacNeil, S.; Tran, A.; Mogil, D.; Bernstein, S.; Ross, E.; Huang, Z. Generating diverse code explanations using the GPT-3 large language model. In Proceedings of the 2022 ACM Conference on International Computing Education Research, Lugano, Switzerland, 7–11 August 2022; Volume 2, pp. 37–39.
15. Viet, T.D.; Markov, K. Using Large Language Models for Bug Localization and Fixing. In Proceedings of the 12th International Conference on Awareness Science and Technology (iCAST), Taichung, Taiwan, 9–11 November 2023; pp. 192–197. [CrossRef]
16. Savelka, J.; Agarwal, A.; An, M.; Bogart, C.; Sakr, M. Thrilled by your progress! Large language models (GPT-4) no longer struggle to pass assessments in higher education programming courses. In Proceedings of the 2023 ACM Conference on International Computing Education Research, Chicago, IL, USA, 7–11 August 2023; Volume 1, pp. 78–92.
17. Dai, W.; Lin, J.; Jin, H.; Li, T.; Tsai, Y.-S.; Gašević, D.; Chen, G. Can Large Language Models Provide Feedback to Students? ACase Study on ChatGPT. In Proceedings of the 2023 IEEE International Conference on Advanced Learning Technologies (ICALT), Orem, UT, USA, 10–13 July 2023. [CrossRef]
18. Yancey, K.P.; Laflair, G.; Verardi, A.; Burstein, J. Rating short l2 essays on the CEFR scale with GPT-4. In Proceedings of the 18th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2023), Toronto, ON, Canada, 13 July–2 May 2023; pp. 576–584.
19. Xiao, C.; Ma, W.; Xu, S.X.; Zhang, K.; Wang, Y.; Fu, Q. From Automation to Augmentation: Large Language Models Elevating Essay Scoring Landscape. *arXiv* **2024**, arXiv:2401.06431.
20. Venter, J.; Coetzee, S.A.; Schmulian, A. Exploring the use of artificial intelligence (AI) in the delivery of effective feedback. *Assess. Eval. High. Educ.* **2024**, 1–21. [CrossRef]
21. Maity, S.; Deroy, A. Human-Centric eXplainable AI in Education. *arXiv* **2024**, arXiv:2410.19822. [CrossRef]
22. Sclar, M.; Choi, Y.; Tsvetkov, Y.; Suhr, A. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting. *arXiv* **2023**, arXiv:2310.11324.
23. Shankar, S.; Zamfirescu-Pereira, J.; Hartmann, B.; Parameswaran, A.; Arawjo, I. Who validates the validators? Aligning LLM-assisted evaluation of LLM outputs with human preferences. In Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology, Pittsburgh, PA, USA, 13–16 October 2024; pp. 1–14.
24. Xu, T.; Wu, S.; Diao, S.; Liu, X.; Wang, X.; Chen, Y.; Gao, J. SaySelf: Teaching LLMs to Express Confidence with Self-Reflective Rationales. *arXiv* **2024**, arXiv:2405.20974.
25. Abbas, N.; Whitfield, J.; Atwell, E.; Bowman, H.; Pickard, T.; Walker, A. Online chat and chatbots to enhance mature student engagement in higher education. *Int. J. Lifelong Educ.* **2022**, *41*, 308–326. [CrossRef]
26. Aleedy, M.; Alshihri, F.; Meshoul, S.; Al-Harthi, M.; Alramlawi, S.; Aldaihani, B.; Shaiba, H.; Atwell, E. Designing AI-powered translation education tools: A framework for parallel sentence generation using SauLTC and LLMs. *PeerJ Comput. Sci.* **2025**, *11*, e2788. [CrossRef]
27. Alsafari, B.; Atwell, E.; Walker, A.; Callaghan, M. Towards effective teaching assistants: From intent-based chatbots to LLM-powered teaching assistants. *Nat. Lang. Process. J.* **2024**, *8*, 100101. [CrossRef]

28. Wang, S.; Xu, T.; Li, H.; Zhang, C.; Liang, J.; Tang, J.; Yu, P.S.; Wen, Q. Large language models for education: A survey and outlook. *arXiv* **2024**, arXiv:2403.18105.
29. Ji, Z.; Lee, N.; Frieske, R.; Yu, T.; Su, D.; Xu, Y.; Ishii, E.; Bang, Y.J.; Madotto, A.; Fung, P. Survey of hallucination in natural language generation. *ACM Comput. Surv.* **2023**, *55*, 1–38. [CrossRef]
30. Huang, L.; Yu, W.; Ma, W.; Zhong, W.; Feng, Z.; Wang, H.; Chen, Q.; Peng, W.; Feng, X.; Qin, B.; et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.* **2025**, *43*, 1–55. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.